Milan House Prices Forecasting

Luca D'Ambrosio

May 15, 2024

1 Overview

To make predictions about house selling' prices in Milan I carried out some feature engineering on which I applied a tuned XGBoost model. I was able to reach a cross-validation' Mean Squared Error of 0.181 on logged house prices and a Mean Absolute deviation of €67,524 on the platform's temporary score.

In the cleaning section, I followed a set of 3 rules. For variables which had too many different values, I made some groups for the unrepresented categories. The groups were made based on my understanding of how certain values are close to each other in the context of house pricing. For missing values, I deemed that these were not missing at random. Hence, I encoded some dummies to indicate missings for each column as they would be an indicator of house prices. Finally, for variables that indicated some kind of ordering (e.g., energy class), I decided to encode them as ordinal numbers and not as dummies. I end up to a total of 221 features.

Because of its known performance for tabular data, I decided to focus on a XGBoost model and chose a set of key parameters to tune. I did combinations of exhaustive and random searches to find the best hyperparameters of my model. I used the mean residual mean squared error over a 10-fold cross validation to evaluate model performance.

2 Data Preparation

2.1 Missing values

When inspecting missing values together with the other conditions of the house, and its price, I decided to assume that missing values were not missing at random. Therefore, I decided to create a dummy variable for each of the columns having

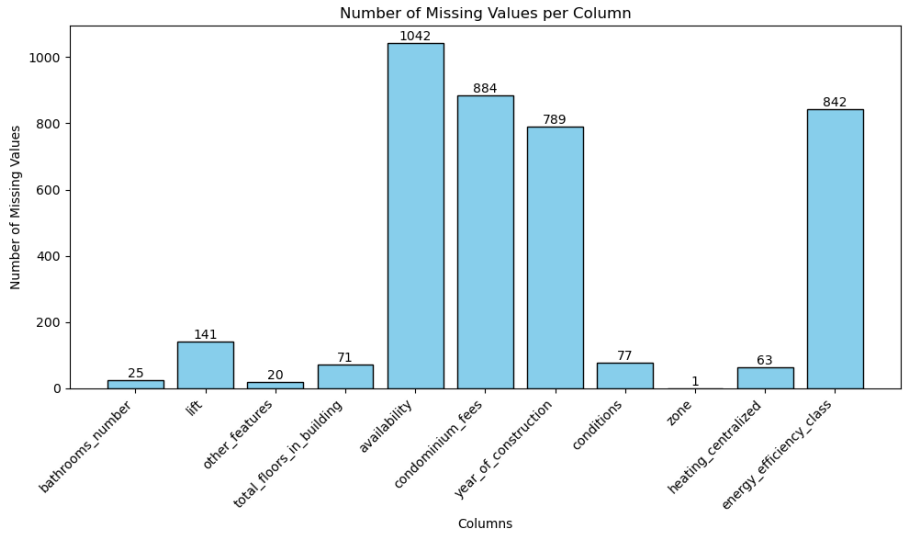

Figure 1: Barplot of the number of missings per feature

missings, indicating whether an observation has a missing for that particular feature, or not. The only exception was the missing in "zone" that I replaced with "città studi", the most represented value for that variable.

2.2 Categorical Variables

- **other_features :** This variable contained a list of features of the exposure of the house and the materials of the windows. I applied some cleaning by de-concatenating values like "pvcexposure" into "pvc exposure" as they were not meant to have a joint meaning. "exposure" was often associated with cardinal points and hence under-represented because of the multiple association of directions. Hence, I decided to split exposure from the directions. The preprocessing for that feature resulted in 43 categories. The most represented category occurred 6569 times, the least represented was 'nan' with 23 occurrences and on average each category had 1723 observations.

- **availability :** I assumed that the availability date could be of interest as it signals sellers that are forward looking to sell. With this strategy they might want to get a higher price than they would if they sold at the last moment. I decided to make 3 groups of future sellers: those in early 2024 (june and pre-june 2024), those in late 2024 (post-fune 2024) and those from 2025 onwards.

- **year_of_construction :** I decided to group the year of construction into decades from 1880 to 2020, create another group for houses of the 18th century

but not post 1880 and finally a group for houses pre 18th century as they might indicate very old houses that have a value attached to their history. I decided to leave the year of construction as a categorical variable as I think that each deceny has its own characteristics and hence should be modeled in a heterogeneous way.

- **zone :** Expecting this feature to have a high importance in determining prices, I didn't want to lose any information from it. I decided to merge under-represented zones (less than 30 observations over the 8,000 observations of the train set) to their most similar zone. To do so, I went to immobiliare.it and checked neighboring zones that were likely to be the most similar. In general, I selected the neighboring zone which is at the same distance from the center (Duomo) and shared the same metro stop. Figure 2 shows an example of merging, the case of QT8 with Monte Stella. I decided to do this process for each zone that had less than 20 observations. I decided to not merge "quintosole - chiaravalle" as I did not find any good similar zone. Additionally, some observations had street coordinates rather than zones and so I manually merged it to the zone it was included in.

- **car_parking :** For the train set, I inspected the cases that had few values. Based on the price and the zone, I was often able to identify such houses on immobiliare.it and understand whether they were strange cases or legit cases of many parking. I decided to create 6 final categories: "missings", "strange cases", 1 in shared, 2 (or more) in shared, 1 in garage, 2 (or more) in garage.

- **lift, conditions,heating :** the variables were already cleaned.

2.3 Ordinal Variables

These are variables which I considered to have some ordering sense, e.g., 3 rooms is better than 2. Therefore, it wouldn't make sense to encode these as dummies, we would lose information.

- **bathrooms_number :** replaced '3+' to 4. For the 25 missings, I assumed that it would more probably be a 0 as this value was not allowed by the site immobiliare.it and it happens that some flats do not have a bathroom. Encoded a dummy for observations that had a missing for that variable.

- **rooms_number :** replaced '5+' to 6

- **total_floors_in_building :** Encoded missings as 0 and decided to replace every floor greater than 9 to 9 to have a total of 10 categories.

Figure 2: Association of similar zones: the case of Monte Stella with QT8

- **condominium_fees :** I set fees greater than €10,000 to missings as they are likely to be errors. I encoded missing values as 0 as they more likely to mean that there are no condominium fees for that house. I transformed the values in hundreds, 4 would include all values in the [400,500) range. Values between 800 and 1000 included were encoded to 8 to have enough observations per category. Values between 1000 and 10000 were encoded to 9 as these are probably legit cases of expensive condiminium fees.

- **floor :** I created two dummies for "mezzanine" and "semi-basement" as they indicate important characteristics of the house and I respectively encoded them in -2 and -1.

- **energy :** Despite being indicated with letters, energy classes have represent some order where 'a' (green) is way better than 'g' (red). I decided to encode missings to 9 which is worst than 'g' (=8) as I assumed that not displaying the energy class is even worst than a 'g'.

2.4 Continuous Variables

I noticed that house prices were skewed so I applied to log transformation to make the data more symmetric and the model less affected by the presence of large values.

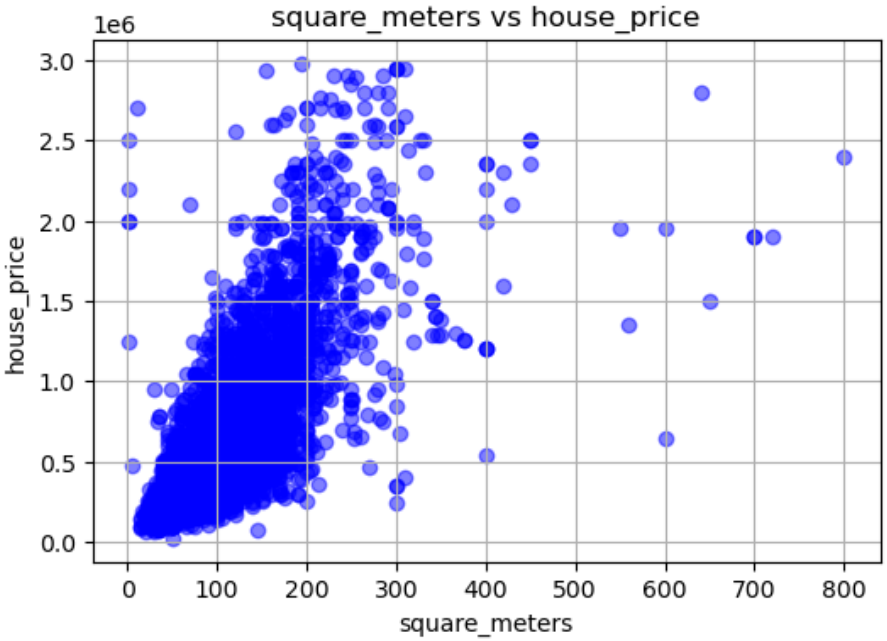

Figure 3: Scatter plot of square meters vs house price

The variable for the house surface is the only continuous feature of this dataset. From Figure 3, it appears that there might be a strong positive linear relationship between square meters and house prices. Further inspecting the data, I did not consider the high values of square meters as outliers. However, I recoded all values inferior of 13 to 13 as these look like typos.

3 Model and Tuning

3.1 XGBoost algorithm

I used the following article to learn about XGBoost: link

Because of its performance results, I chose to apply an Extreme Gradient Boosting (XGBoost) algorithm. It is a specific implementation of the gradient boosting algorithm (GBR) that is designed to be computationally efficient and highly effective.

Both of these models build on the idea to iteratively add models to the ensemble, each of which is trained to correct the errors made by the previous models, namely ensemble learning models. This process is driven by the gradient of the loss function, which is used to update the model parameters in each iteration. When the model

iteratively fits weak learners on the RSS of the previous tree, it implicitly estimates the gradient of the loss. To output final predictions, the algorithm accumulates the predictions of the many weak learners trained in that fashion.

The objective function of the XGBoost differs from the objective function of GBR by incorporating regularization elements in the objective function to prevent overfitting. The regularization term is the sum of the regularization terms for each node in the tree. At each iteration the first and second derivatives of the loss functions are calculated with respect to the predicted output. The best tree is then the tree that minimizes the objective function given the gradient, the Hessian and for given regularizing hyperparameters.

Compared to GBR, it also has a more sophisticated tree construction method that allows for deeper trees and more complex splits that I used in my tuning. The regularizing term in the objective function reduces both overfitting and computation power required.

3.2 Parameters tuning

To avoid extensive waiting time, I used a combination of RandomizedSearch cross-validation later on refined by GridSearch CV. I used 5 folds cross-validation and towards the end to get more precision a 10 folds one. In this regression setting, I decided to focus on tuning the following parameters:

- **learning_rate =0.01 :** Scales the contribution of each tree in the ensemble. A lower learning rate makes the model more robust by shrinking the contribution of each tree.
- **n_estimators =26,000:** The number of trees in the ensemble.
- **n_estimators =3:** The maximum depth of each tree.
- **subsample =0.6:**The fraction of training instances (samples) to be randomly sampled for each tree. It introduces stochasticity into the training process, which can help prevent overfitting by providing diversity in the data used for each tree.
- **colsample_bytree =0.8:** The fraction of features to be randomly sampled for each tree.
- **colsample_bylevel =0.8:** The fraction of features to be randomly sampled at each split of the tree.

- **colsample_bytree =0.8:** The fraction of features to be randomly sampled for each level.
- **colsample_bynode =0.8:** The fraction of features to be randomly sampled for each node.

Interestingly, the model was performing better leveraging on a (very) large number of trees balanced by introducing some randomness in the features used by the trees to make each tree a (very) weak learner. I was also surprised by the model selecting a max_depth of 3 which I would have believed to be too small. These results show that the model selected the strength of XGBoost being its ability to train a large amount of weak learners without overfitting.

With this hyperparameter tuning I was able to achieve a 10-fold Cross-Validation score of 0.181 (mean residual mean squared error on the log-transformed y) on a 80% split of the train data and mean deviation score of 70,131 on the unseen 20% of the validation set.